

PROJECT DESIGN REPORT

Lightweight Temporal and Confidence-Aware Lane Detection

For unstructured roads under faded markings, temporary occlusion and poor visibility

Shared Research and Implementation Reference

1. Executive Summary

This project builds a camera-based lane perception module for an autonomous vehicle or ADAS prototype. The system reads a forward-facing road video, detects the left and right lane boundaries, uses recent frames to reduce temporary loss of a lane, estimates a reliability score, and outputs lane geometry that a later path-planning module could consume.

Final problem in one sentence
Single-frame lane detectors can become fragmented or unstable when markings are faded, broken, shadowed or briefly hidden; the proposed model uses short-term video memory and confidence estimation to provide more stable, honest lane predictions.

What this project is - and is not

It is	It is not
A Computer Vision and deep-learning lane perception project	A complete self-driving car
A comparison between a single-frame baseline and a temporal model	An LLM or RAG application
A research prototype evaluated on video datasets	A safety-certified production system
A source of lane coordinates, centre, curvature and confidence	A claim that the vehicle can safely drive in every condition

Recommended final title

Working title
Lightweight Temporal and Confidence-Aware Lane Detection for Unstructured Roads Under Adverse Visibility

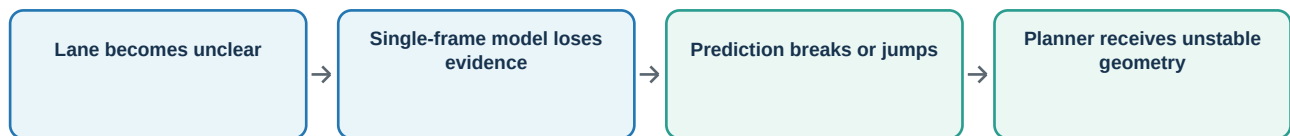
Publication potential

A working project is achievable with a staged team workflow. A publishable paper is possible only if the proposed model produces a measurable improvement over reproduced baselines, includes fair experiments and ablation studies, and survives a final literature and patent novelty check. Publication cannot be guaranteed at the idea stage.

2. Problem Definition and Motivation

Lane perception is a basic input to lane keeping, path planning and driver-assistance systems. In clean daytime highway scenes, painted lanes are usually visible. Real roads are more difficult: paint fades, markings break, shadows resemble lines, vehicles cover parts of the lane, glare reduces contrast, and curves change the apparent shape.

Observed failure chain



Research problem statement

Formal statement

Existing lane-detection systems often process frames independently and may produce fragmented, flickering or incorrect lane boundaries when lane evidence is degraded or temporarily occluded. The project investigates whether a lightweight temporal memory mechanism can improve lane continuity and stability while preserving real-time speed, and whether calibrated confidence can identify predictions that should not be trusted.

Scope boundaries

- Primary target: faded, broken and temporarily occluded painted lane markings.
- Conditions: normal, curve, shadow, night/low light, crowded traffic and wet-road visibility where data is available.
- Input: monocular RGB video from a front-facing camera.
- Output: lane mask or lane points, fitted left/right curves, lane centre, curvature, vehicle offset estimate and confidence.
- Out of initial scope: completely unmarked roads with no defensible lane ground truth, real-vehicle actuation, LiDAR/radar fusion, and legal safety certification.

Plain-language explanation

A human driver does not forget a lane merely because another vehicle hides it for one second. The proposed model similarly keeps a short memory of recent frames. It should continue a lane only when recent visual evidence supports it, and lower its confidence when the estimate becomes uncertain.

3. Literature Synthesis and Defensible Research Gap

The supplied papers and established public work show that lane detection itself is not a new problem. Flexible curve models, deep lane segmentation, lightweight networks and video memory already exist. Therefore the proposal must not claim to be the first lane detector or the first temporal lane detector.

Work	What it contributes	Relevant limitation/gap
Catmull-Rom spline lane detection (1998)	Flexible mathematical curves; maximum-likelihood matching; coarse-to-fine search	Needs useful image edges; small visual test; no modern quantitative benchmark
Roadside sensor resilience (2026)	Infrastructure sensing and fusion for adverse weather and sensor failure	Not a dedicated camera-only lane solution; infrastructure and communication are required
Safety-aware multitask perception (2026)	YOLOv8/C3TR model for anomalies, drivable area and lanes on Indian roads	Lane output degrades when markings are unclear; lane task is weaker than drivable-area performance
CULane / SCNN	Large challenging lane benchmark with normal, shadow, curve, crowded and night cases	Frames are from Beijing; not specifically an Indian adverse-road benchmark
VIL-100 / MMA-Net	Video instance lane dataset and temporal memory aggregation	Shows that temporal lane detection already exists; it becomes a baseline/related work, not the novelty claim
TwinLiteNet/TwinLiteNet +	Very lightweight lane and drivable-area segmentation	Primarily image-based baseline; project must prove any temporal addition improves difficult video cases

Defensible candidate gap

There is room for a carefully evaluated lightweight system that combines short video memory, lane continuity objectives and calibrated reliability for difficult/unstructured road scenes. This is a candidate gap, not a guaranteed global novelty claim. The final paper must compare the exact architecture and claims against the closest recent work.

Proposed research questions

- RQ1: Does short-term temporal memory improve lane F1/IoU and continuity in occlusion, shadow, curve and low-light subsets?
- RQ2: Does the temporal model reduce frame-to-frame lane flicker compared with the single-frame baseline?
- RQ3: Can confidence calibration identify unreliable predictions without rejecting too many correct ones?
- RQ4: Can these gains be achieved while maintaining practical real-time inference speed?

4. Functional Requirements and Success Criteria

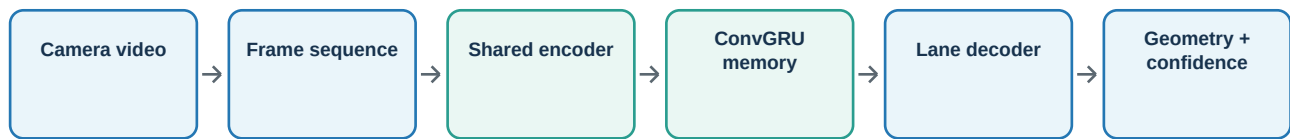
ID	Requirement	Acceptance evidence
FR-1	Accept an MP4 or camera stream	Video is decoded without dropping the complete run
FR-2	Detect lane pixels/points per frame	Overlay and stored lane coordinates are produced
FR-3	Use recent frames	Temporal state or frame sequence is active during inference
FR-4	Estimate left/right lane geometry	Fitted curves and lane centre are returned
FR-5	Produce reliability status	Calibrated score and high/medium/low state are logged
FR-6	Measure speed and accuracy	F1, IoU, stability, latency and FPS report is generated
FR-7	Save a processed video	Output video contains overlay and status panel

Research success is comparative

The project succeeds as a prototype when the complete pipeline works. The research hypothesis succeeds only if the temporal version consistently improves difficult-condition metrics over the same baseline under the same train/test protocol.

Area	Suggested target (not a promised result)
Accuracy	Positive F1/IoU improvement on difficult subsets
Continuity	Fewer gaps and lower frame-to-frame instability
Confidence	Lower confidence on actual failure cases; acceptable calibration error
Efficiency	At least 20-30 FPS on the declared test GPU, or honest latency reporting
Scientific quality	Video-level split, baseline reproduction, ablations and failure analysis

5. Proposed Technical Architecture



5.1 Frame sequence

The loader supplies a short ordered sequence such as frames $t-2$, $t-1$ and t . Frames must come from the same video and remain in time order.

5.2 Shared visual encoder

A lightweight convolutional encoder extracts road textures, edges, curves and lane-marking features from each frame. The recommended practical route is to reuse the encoder and lane head of an open-source lightweight baseline rather than inventing an entire network before the pipeline works.

5.3 Temporal memory

A ConvGRU receives current visual features and the hidden state from recent frames. It updates memory spatially, preserving useful lane evidence while allowing the state to change as the road curves. It is inserted at a reduced feature resolution to control compute cost.

5.4 Lane decoder

The decoder upsamples the temporally enhanced feature map into a lane probability mask. A threshold converts probabilities into a binary mask; post-processing then extracts lane components and points.

5.5 Geometry and reliability

Lane points are fitted with a polynomial or spline to generate left/right curves. The system estimates lane centre, approximate curvature and lateral image offset. Reliability combines calibrated lane probability, continuity and agreement with recent frames. A low score tells the downstream module not to rely blindly on the output.

Operational rule
The model may bridge a short temporary occlusion when recent evidence is strong. It must not indefinitely hallucinate a lane after evidence disappears. Memory reset and confidence decay are therefore required.

6. Dataset and Annotation Plan

Dataset	Use in project	Important note
CULane	Large baseline training/evaluation; difficult categories such as shadow, curve, crowded and night	Large download; begin with an approved subset, then scale
VIL-100	Primary temporal/video experiment with consecutive annotated frames	Best fit for validating temporal memory; not an Indian-road dataset
IDD	Cross-domain unstructured Indian road analysis	Broader semantic scene dataset; verify lane-label suitability before training
IDD-AW	Adverse-weather Indian scene analysis	RGB/NIR semantic dataset; verify dedicated lane labels and licensing
Small team dataset	Indian road qualitative test and optional fine-tuning	Requires safe collection, privacy handling and consistent annotation policy

Recommended staged data strategy

- Stage A: make the baseline run on a small CULane/VIL-100 sample.
- Stage B: train and evaluate temporal behavior on VIL-100 or consecutive CULane frames.
- Stage C: evaluate difficult-condition subsets separately.
- Stage D: collect 20-30 short Indian-road clips only if needed; annotate 300-500 carefully selected frames first.
- Stage E: reserve complete routes/videos for testing. Never split neighbouring frames from one clip across training and testing.

Annotation policy

- Use CVAT or Label Studio and define whether labels are thin polylines or segmentation masks.
- Write rules for dashed lines, intersections, occluded sections, road edges and ambiguous frames before annotation begins.
- Two members should review a sample of annotations; disagreements become documented rules.
- Mark scenario tags: normal, curve, shadow, night, crowded, occlusion, wet road and severely unclear.
- If videos are shared publicly, blur faces/number plates as required and document consent, ownership and dataset licence.

7. Implementation Blueprint

Build the system in two layers: a guaranteed MVP and a research extension. This keeps the project demonstrable even if model training takes longer than expected.

Stage	Implementation work	Deliverable
1. Environment	Python, PyTorch, OpenCV, augmentation library, Git repository and reproducible environment file	One command/notebook launches inference
2. Baseline	Run pretrained lane model on images, then MP4 frames; save masks and overlays	Baseline video and initial FPS
3. Evaluation	Implement F1, precision, recall, IoU and per-condition result table	Reproduced baseline report
4. Sequence loader	Return ordered frame windows and matching masks without crossing video boundaries	Verified batches of 3-5 frames
5. Temporal model	Insert ConvGRU between encoder and lane decoder; manage hidden-state reset	Trainable temporal network
6. Losses	Segmentation loss plus continuity and temporal-consistency terms	Ablatable training configuration
7. Geometry	Extract lane points, reject tiny components, fit curves and estimate centre/curvature	Structured lane output
8. Confidence	Calibrate probabilities; combine with continuity and temporal agreement	Reliability score and warning
9. Demo	Overlay lanes, centreline, confidence, condition label, latency and FPS	Processed MP4 or live dashboard

Core model pseudocode

```
for each video sequence:
    reset temporal memory
    for each ordered frame:
        features = encoder(frame)
        memory = ConvGRU(features, memory)
        lane_probability = decoder(memory)
        lane_mask = postprocess(lane_probability)
        geometry = fit_lane_curves(lane_mask)
        confidence = calibrate(probability, continuity, temporal_agreement)
        save(overlay, geometry, confidence)
```

8. Training Design

Training begins only after the baseline inference and metrics are verified. The first experiment should freeze or partially freeze the pretrained encoder, train the temporal module and decoder, then gradually unfreeze layers if necessary.

Input and augmentation

- Input size: start with the baseline's native resolution (commonly around 640-wide) rather than arbitrary resizing.
- Sequence length: begin with 3 frames; compare 1, 3 and 5 in an ablation.
- Apply the same geometric augmentation to every frame and mask in a sequence.
- Use brightness, contrast, blur, shadow and limited rain-like augmentation carefully; keep an untouched real test set.
- Do not reverse time or randomly shuffle frames inside a sequence.

Loss design

Loss	Role	Caution
Weighted BCE/Focal	Lane vs background pixel classification	Thin lanes create class imbalance
Dice	Encourages overlap of predicted and labelled lane masks	Not sufficient alone for topology
Continuity/topology	Penalizes broken lane structures	Must not join unrelated markings
Temporal consistency	Reduces unnecessary changes across frames	Account for camera/vehicle motion; do not force identical masks

Conceptual total loss

Training objective
$L_{\text{total}} = L_{\text{segmentation}} + \lambda_c * L_{\text{continuity}} + \lambda_t * L_{\text{temporal}}$. The lambda values must be selected on validation data and reported. The paper should include an ablation showing the effect of each term.

Training safeguards

- Fix random seeds and save exact configurations.
- Use early stopping and store the best validation checkpoint.
- Track training/validation loss separately to detect overfitting.
- Keep the baseline training recipe unchanged when comparing models.
- Report the GPU, input resolution, batch size, sequence length, optimizer, epochs and inference protocol.

9. Output Contract for the Autonomous-Vehicle Pipeline

The lane model does not directly drive a real car. It produces structured perception information for a downstream planner/controller or for a simulator demonstration.

Example output record

```
{
  "frame_id": 148,
  "left_lane_points": [[210,350], [225,300], [240,250]],
  "right_lane_points": [[430,350], [410,300], [395,250]],
  "lane_center_x": 320,
  "curvature": "left_curve",
  "confidence": 0.82,
  "status": "reliable"
}
```

Field	Meaning	Downstream use
Lane points/curves	Estimated left and right lane geometry	Build local lane corridor
Lane centre	Midpoint between valid lane boundaries	Estimate desired path
Curvature	Approximate bend direction and magnitude	Anticipate a curved path
Offset	Difference between camera/vehicle centre and lane centre	Lateral correction input
Confidence/status	Reliability of current geometry	Use, degrade reliance or trigger fallback

Confidence behavior

- High: strong lane probability, continuous curve and agreement with recent frames.
- Medium: partial visibility or short temporal interpolation; usable only with caution.
- Low: weak evidence, inconsistent geometry or memory too old; downstream module receives an unreliable flag.
- No lane: no defensible estimate; the research prototype reports failure instead of inventing a path.

Safety disclaimer

In the student prototype, a low-confidence output should trigger only a displayed/logged warning or simulator behavior. Real braking, steering or driver-handover logic requires separate safety engineering and validation.

10. Evaluation and Experiment Plan

A publishable result needs more than an attractive overlay video. Quantitative metrics, controlled comparisons and honest failure cases are required.

Metric	Question answered
Precision	How many predicted lane pixels/lanes were correct?
Recall	How much of the labelled lane was recovered?
F1 score	What is the balance between precision and recall?
Lane IoU	How strongly do predicted and true lane regions overlap?
Continuity/gap score	How fragmented are the predicted lanes?
Temporal stability	How much does aligned lane output flicker across adjacent frames?
ECE or reliability diagram	Does stated confidence match observed correctness?
FPS/latency	Can the model run near real time on declared hardware?
Parameters/FLOPs	What computational cost was added?

Required comparisons

Experiment	Purpose
Single-frame baseline	Establish reproducible starting point
Baseline + ConvGRU	Measure temporal memory contribution
Baseline + continuity loss	Measure structural objective contribution
Baseline + ConvGRU + all losses	Evaluate complete proposed model
Sequence length 1 vs 3 vs 5	Check benefit/cost of additional history
Normal vs shadow vs curve vs night vs occlusion	Reveal where the method actually helps or fails

Statistical discipline

- Split by video/route, not by randomly mixing adjacent frames.
- Use the same test set and evaluation code for every model.
- Repeat training with multiple seeds if compute permits; report mean and variation.
- Show examples where the temporal model helps and where memory causes errors.
- Do not claim adverse-weather robustness from synthetic augmentation alone.

11. Software Design and Demonstration

The simplest credible demonstration is an offline video pipeline. A web dashboard is optional presentation polish, not the research contribution.

Recommended repository structure

```
lane-temporal-project/  
|-- configs/          # training and evaluation settings  
|-- data/             # loaders, splits and annotation conversion  
|-- models/           # baseline, ConvGRU and lane decoder  
|-- losses/           # segmentation, continuity and temporal losses  
|-- inference/        # video processing, post-processing, geometry  
|-- evaluation/       # metrics, subset analysis and plots  
|-- app/              # optional Streamlit demonstration  
|-- outputs/          # checkpoints, masks, videos and CSV results  
|-- train.py  
|-- evaluate.py  
|-- infer_video.py  
|-- requirements.txt  
|-- README.md
```

Demonstration screen

Panel	Displayed information
Video	Original frame and lane overlay
Geometry	Left/right curves, lane centre and approximate vehicle offset
Reliability	Confidence score and reliable/caution/unreliable state
Performance	Frame number, latency and FPS
Diagnostics	Current condition tag and optional previous-frame comparison

Optional simulator extension

If time permits, CARLA can provide a controlled demonstration of clear, rain, fog or low-light scenarios. The model output may be logged or used for a simple simulated lane-centre controller. This is an extension; it should not delay the core dataset experiments.

12. Project Schedule and Team Work

Weeks	Focus	Exit condition
1-2	Literature matrix, dataset access, environment and baseline inference	One baseline video works
3-4	Metrics, baseline reproduction and failure categorization	Baseline result table complete
5-7	Sequence loader and ConvGRU integration	Temporal model trains and infers
8-9	Continuity/temporal loss and confidence calibration	Full model output available
10-11	Condition-wise experiments and ablations	Comparison tables complete
12	Video demonstration and structured output	End-to-end prototype ready
13-15	Repeat experiments, failure analysis and figures	Research evidence frozen
16	Paper/report writing and internal review	Submission-ready draft

Suggested role split

Role	Primary ownership
Research and documentation	Literature matrix, problem/gap, experiment protocol, citations, paper writing
Data	Dataset download, sequence indexing, annotation conversion, quality checks and splits
Model	Baseline reproduction, ConvGRU integration, losses and training
Evaluation/demo	Metrics, ablations, plots, video inference, UI and packaging

All members should understand the common core

Even with separate ownership, every member should be able to explain the problem, data split, baseline, proposed modification, metrics and main result. Git branches, weekly integration and experiment logs prevent the project from becoming four disconnected parts.

13. Risks, Fallbacks and Common Mistakes

Risk	Why it matters	Fallback/mitigation
Dataset too large	Training/download blocks progress	Start with a documented subset; scale after pipeline works
No appropriate Indian temporal lane labels	Claim cannot be quantitatively tested	Use VIL-100/CULane for temporal metrics; annotate a small held-out Indian video set
ConvGRU lowers FPS	Real-time claim becomes weak	Use reduced feature resolution, fewer channels or a temporal-shift/EMA baseline
Memory propagates an old wrong lane	Temporal model can be worse after occlusion	Confidence decay, reset rules and occlusion-length analysis
Pretty demo but weak science	Not enough for publication	Prioritize metrics, ablations and video-level split
Overclaiming novelty	Reviewers can reject the paper	Use precise claims and compare with VIL-100/MMA-Net and recent temporal work
Test leakage	Artificially high scores	Split complete videos/routes before creating frame windows

MVP fallback

If learned temporal training is temporarily blocked, build a complete baseline video system and compare simple temporal smoothing or optical-flow-assisted mask propagation. This keeps the project operational, but it should be presented as an engineering baseline, not automatically as the final novel method.

Do not add unrelated components

LLM and RAG are not required for real-time lane perception and should not be added merely to increase buzzwords. They neither create lane coordinates nor make the detector safer by themselves. The focused project already covers Computer Vision, deep learning, sequence modelling, confidence calibration and real-time inference.

14. Research Paper Plan

Section	Content
Abstract	Problem, proposed temporal-confidence approach, datasets, key numerical result and conclusion
Introduction	Why unstable lane perception matters; exact gap and contributions
Related Work	Classical curves, image-based deep models, lightweight models, video lane detection and uncertainty
Method	Baseline, sequence input, ConvGRU, decoder, losses, confidence and geometry
Experimental Setup	Datasets, video split, augmentation, hardware, hyperparameters and metrics
Results	Baseline comparison, subset tables, speed and calibration
Ablation	Temporal module, loss terms, sequence length and confidence design
Failure Analysis	Long occlusion, missing paint, intersection, night glare and wrong-memory cases
Limitations	Dataset/domain limits, monocular ambiguity, no real-car safety validation
Conclusion	What was actually demonstrated and next steps

Contribution wording

Safe claim template

We present and evaluate a lightweight temporal lane-perception framework that augments a reproducible single-frame baseline with short-term spatial memory, continuity-aware optimization and calibrated reliability. We study its accuracy, temporal stability and efficiency under difficult video conditions.

Claims to avoid

- First-ever lane detection system.
- First-ever use of video or temporal memory for lane detection.
- Works on all Indian roads or all weather conditions.
- Safe for real autonomous vehicles without vehicle-level safety testing.
- Patentable or globally novel before a professional prior-art search.

15. References and Starting Resources

- Wang, Y., Shen, D., and Teoh, E. K. (1998). Lane Detection Using Catmull-Rom Spline. IEEE Intelligent Vehicles.
- Lamichhane, B. R. et al. (2026). Enhancing autonomous vehicle resilience: Roadside sensor networks for robust perception and decision-making in challenging environments. Array, 30, 100784.
- Selvarajan, N. P., Megalingam, R. K., and Shehsaath, V. S. (2026). Safety-aware transformer-enhanced unified multi-task perception for road anomaly detection, drivable area segmentation and lane estimation. Scientific Reports. DOI: 10.1038/s41598-026-60419-8.
- Pan, X. et al. (2018). Spatial As Deep: Spatial CNN for Traffic Scene Understanding. AAAI. CULane project: <https://xingangpan.github.io/projects/CULane.html>
- Zhang, Y. et al. (2021). VIL-100: A New Dataset and A Baseline Model for Video Instance Lane Detection. ICCV. Code/data: <https://github.com/yujun0-0/MMA-Net>
- Che, Q. H. et al. (2023). TwinLiteNet: An Efficient and Lightweight Model for Driveable Area and Lane Segmentation in Self-Driving Cars. IEEE. Code: <https://github.com/chequanghuy/TwinLiteNet>
- Varma, G. et al. (2019). IDD: A Dataset for Exploring Problems of Autonomous Navigation in Unconstrained Environments. WACV. Dataset: <https://idd.insaan.iiit.ac.in/>
- Shaik, A. et al. (2023/2024). IDD-AW: A Benchmark for Safe and Robust Segmentation of Drive Scenes in Unstructured Traffic and Adverse Weather. Dataset: <https://iddaw.github.io/>

Recommended literature-search groups

Search group	Example query
Temporal lane detection	video lane detection temporal consistency ConvGRU lane
Difficult conditions	lane detection occlusion shadow night rain faded markings
Unstructured roads	lane detection unstructured Indian roads missing markings
Reliability	uncertainty calibration confidence estimation lane detection
Efficiency	lightweight real-time video lane detection embedded
Prior art	patent lane detection temporal confidence autonomous vehicle

Final note

This report is an implementation and research blueprint. Before freezing the paper title or claiming novelty, update the related-work matrix with the latest peer-reviewed papers and patent results, then state only the difference that your experiments actually demonstrate.